MINI-PROJECT REPORT

Student Placement Prediction

Predicting campus placement and student readiness using machine learning

Team members

Apeksha
Ananya K
Ashritha S Suvarna

November 2026

Abstract

This project predicts whether a student will be placed, and gives the chance of placement as a readiness percentage. We used a public dataset of 10,000 students (train, validation and test files) with ten attributes such as CGPA, aptitude score, 10th and 12th marks, projects, internships and placement training. The data was checked and cleaned (43 repeated rows removed, no missing values or outliers) and explored with statistics and graphs; aptitude score, school marks, extracurricular activities and placement training showed the strongest links with placement.

Logistic Regression and Random Forest were compared on the validation file, and Logistic Regression was chosen because Random Forest memorized the training data. On 1,500 unseen test students the model reached 79.1% accuracy, and its readiness percentage matched real outcomes: 14.8% of students in the Low band, 54.2% in the Medium band and 81.8% in the High band were placed.

We also collected our own survey of 57 students, which is reported separately because it could not be used to test the model. The trained model is saved as one file and the full work is kept on GitHub.

Keywords: campus placement, classification, logistic regression, exploratory data analysis, readiness

Table of Contents

Chapter 1: Introduction . . . 5

1.1 Background and Motivation . . . 5
1.2 Problem Statement . . . 5
1.3 Objectives . . . 5
1.4 Dataset Description . . . 5
1.5 Organisation of the Report . . . 6

Chapter 2: Data Preprocessing and Exploratory Data Analysis . . . 7

2.1 Dataset Understanding . . . 7
2.2 Data Cleaning and Missing Value Handling . . . 7
2.3 Data Preprocessing . . . 8
2.4 Exploratory Data Analysis (EDA) . . . 9
2.5 Key Insights and Observations . . . 13

Chapter 3: Model Building . . . 15

3.1 Approach . . . 15
3.2 Inputs and Target . . . 15
3.3 The Two Models . . . 15
3.4 Training and Validation . . . 15
3.5 Saving the Model . . . 16
3.6 Placement Chance and Readiness Bands . . . 16

Chapter 4: Results and Evaluation . . . 17

4.1 Test Accuracy . . . 17
4.2 Confusion Matrix . . . 17
4.3 What the Model Learned . . . 18
4.4 Is the Readiness Percentage Meaningful? . . . 19
4.5 Example Predictions . . . 19
4.6 Summary of Results . . . 20

Chapter 5: Our Own Student Survey . . . 21

5.1 What We Collected . . . 21
5.2 What the Survey Shows . . . 21
5.3 Why the Survey Was Not Used with the Model . . . 22

Chapter 6: Conclusion, Limitations and Future Work . . . 23

6.1 Conclusion . . . 23
6.2 Limitations . . . 23
6.3 Future Work . . . 23

Appendix: Project Files, How to Run and How to Use the Model . . . 24

List of Figures

Figure 1: Placed vs NotPlaced (bar chart) . . . 10
Figure 2: Spread of each column (histograms) . . . 11
Figure 3: Placed vs NotPlaced for each column (box plots) . . . 11
Figure 4: Placement rate for selected columns (bar charts) . . . 12
Figure 5: CGPA against aptitude score (scatter plot) . . . 12
Figure 6: Correlation heatmap . . . 13
Figure 7: Right and wrong guesses on the test file (confusion matrix) . . . 17
Figure 8: How much each column matters to the model . . . 18
Figure 9: Readiness index by internship and by technical club membership . . . 22

List of Tables

Table 1: The three data files . . . 5
Table 2: Columns of the dataset . . . 6
Table 3: First five rows of train.csv . . . 7
Table 4: Missing values and repeated rows . . . 8
Table 5: Outlier check (IQR rule) . . . 8
Table 6: Encoding of text columns . . . 9
Table 7: Descriptive statistics of the numeric columns . . . 9
Table 8: Placement rate by group . . . 12
Table 9: Accuracy of the two models . . . 15
Table 10: Confusion matrix on the test file . . . 18
Table 11: Importance of each column . . . 19
Table 12: Real placement rate in each readiness band (test file) . . . 19
Table 13: Ten random test students . . . 20
Table 14: Two made-up students . . . 20
Table 15: Summary of results . . . 20
Table 16: Survey summary (57 students) . . . 21
Table 17: Files in the Placement-Model repository . . . 24

Chapter 1: Introduction

1.1 Background and Motivation

Campus placement decides the first career step of most engineering students, and colleges are judged partly on how many of their students get placed. Students usually learn whether they are "ready" only when placement season starts, which is often too late to improve.

Placement depends on several things together: academic marks (CGPA, 10th and 12th marks), skills (aptitude and soft skills), and experience (internships, projects, workshops, extracurricular activities and placement training). Looking at one factor alone, such as CGPA, does not give a fair picture.

A data-driven approach can learn from the results of past students and show which factors matter most. A college can then spot students who need extra help early and guide them on what to improve.

1.2 Problem Statement

Given a student's academic record, skill scores and experience, predict whether the student will be placed (Placed) or not placed (NotPlaced), and give the chance of placement as a percentage.

1.3 Objectives

1. To clean and explore the placement dataset using statistics and graphs, in order to find which student factors are linked to placement.
2. To train a machine learning model that predicts placement, and to measure its accuracy on unseen test data (target: at least 75% accuracy).

1.4 Dataset Description

The data is the public dataset *Campus_Recruitment_CSV* by Krooz on Hugging Face (folder `raw_datasets`). It is released as free-to-use data. The page does not say which college the students came from, so the results describe this dataset and may differ for other colleges. The data is split into three files, which are used as shown in Table 1.

Table 1: The three data files

File	Students	Used for
train.csv	7,225	The model learns from it
validation.csv	1,275	Practice test to choose the better model
test.csv	1,500	Final check on unseen students

In total there are 10,000 student records. Each file has 11 columns: ten attributes (features) and one target, described in Table 2.

Table 2: Columns of the dataset

Column	Type	Meaning
CGPA	Decimal	Cumulative grade point average (6.5 to 9.1)
Internships	Whole number	Number of internships (0 to 2)
Projects	Whole number	Number of projects (0 to 3)
Workshops/Certifications	Whole number	Workshops or certifications done (0 to 3)
AptitudeTestScore	Whole number	Aptitude test score (60 to 90)
SoftSkillsRating	Decimal	Soft skills rating (3.0 to 4.8)
ExtracurricularActivities	Yes / No	Takes part in extracurricular activities
PlacementTraining	Yes / No	Attended placement training
SSC_Marks	Whole number	10th standard marks (55 to 90)
HSC_Marks	Whole number	12th standard marks (57 to 88)
PlacementStatus	Placed / NotPlaced	**Target:** the result we want to predict

The target is a two-class label, so this is a classification problem.

1.5 Organisation of the Report

The rest of the report is organised as follows:

- **Chapter 2** describes the data, the cleaning and preprocessing steps, and the exploratory analysis with statistics and graphs.
- **Chapter 3** explains how the two models were built, trained and compared, and why one was chosen.
- **Chapter 4** presents the results on unseen test data and checks whether the readiness percentage is meaningful.
- **Chapter 5** reports our own student survey and why it was kept separate from the model.
- **Chapter 6** gives the conclusion, the limitations and ideas for future work.
- The **Appendix** lists the project files on GitHub and shows how to run the notebook and use the saved model.

Chapter 2: Data Preprocessing and Exploratory Data Analysis

2.1 Dataset Understanding

The three files were loaded with pandas and their size was checked.

```
train = pd.read_csv("data/train.csv")
val   = pd.read_csv("data/validation.csv")
test  = pd.read_csv("data/test.csv")
print("train     :", train.shape)
print("validation:", val.shape)
print("test      :", test.shape)
```

Output:

```
train     : (7225, 11)
validation: (1275, 11)
test      : (1500, 11)
```

Train has 7,225 rows and 11 columns. Validation and test have the same 11 columns, so all three files can be processed in the same way.

Seven columns are numbers (CGPA and SoftSkillsRating are decimals; Internships, Projects, Workshops/Certifications, AptitudeTestScore, SSC_Marks and HSC_Marks are whole numbers) and three are text (ExtracurricularActivities, PlacementTraining and PlacementStatus). The text columns need to be converted to numbers before modelling (see 2.3). Table 3 shows the first five rows of train.csv.

Table 3: First five rows of train.csv

CGPA	Intern-ships	Pro-jects	Work-shops	Aptitude	Soft skills	Extra-curr.	Training	SSC	HSC	Status
7.7	1	1	0	69	4.0	No	No	55	69	NotPlaced
7.6	2	3	1	80	4.3	Yes	Yes	57	70	NotPlaced
7.0	1	1	1	64	4.1	No	No	58	62	NotPlaced
7.9	1	3	2	90	4.8	Yes	Yes	82	80	Placed
6.7	0	2	1	71	3.7	No	No	55	67	NotPlaced

Each row is one student. For example, the fourth student has an aptitude score of 90, placement training and extracurricular activities, and is Placed, while the first student, with a similar CGPA but a lower aptitude score, is NotPlaced.

2.2 Data Cleaning and Missing Value Handling

Missing values and repeated rows

```
for name, d in [("train", train), ("validation", val), ("test", test)]:
    print(name, "-> missing values:", int(d.isna().sum().sum()),
          "| repeated rows:", int(d.duplicated().sum()))
```

Table 4: Missing values and repeated rows

File	Missing values	Repeated rows
train	0	43
validation	0	0
test	0	0

There are no empty cells in any file, so no filling or deleting of values was needed. Train has 43 rows that are exact copies of other rows. A repeated row would make the model see the same student twice and look better than it is, so they were removed.

```
train = train.drop_duplicates().reset_index(drop=True)
# Rows removed from train: 43 | train now: (7182, 11)
```

Only train was de-duplicated. Validation and test were left exactly as they are, because they are used to judge the model.

Inconsistent data

The text columns were checked for spelling or case differences. ExtracurricularActivities and PlacementTraining contain only Yes and No, and PlacementStatus contains only Placed and NotPlaced, so no correction was needed.

Outliers

An outlier is a value far from the rest. It was checked with the IQR rule: a value is an outlier if it is below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR. Internships, Projects and Workshops/Certifications have only a few possible values (0 to 3), so this rule is not meaningful for them and they were skipped.

Table 5: Outlier check (IQR rule)

Column	Outliers found	Lower limit	Upper limit
CGPA	0	6.20	9.40
AptitudeTestScore	0	52.00	108.00
SoftSkillsRating	0	2.95	5.75
SSC_Marks	0	30.50	106.50
HSC_Marks	0	43.00	107.00

Every value lies inside its limits, so no outlier was found and no row was removed.

2.3 Data Preprocessing

Encoding

Models can only work with numbers, so the three text columns were converted to 1 and 0. The same function was applied to train, validation and test.

```
def prepare(d):
    d = d.copy()
    d["ExtracurricularActivities"] = d["ExtracurricularActivities"].map({"Yes": 1, "No": 0})
    d["PlacementTraining"]         = d["PlacementTraining"].map({"Yes": 1, "No": 0})
    d["PlacementStatus"]           = d["PlacementStatus"].map({"Placed": 1, "NotPlaced": 0})
    return d

train = prepare(train); val = prepare(val); test = prepare(test)
```

Table 6: Encoding of text columns

Column	Before	After
ExtracurricularActivities	Yes / No	1 / 0
PlacementTraining	Yes / No	1 / 0
PlacementStatus (target)	Placed / NotPlaced	1 / 0

A check after encoding found 0 blank cells, which means every value was converted correctly.

Standardization (scaling)

The columns use very different scales, for example CGPA is about 6.5 to 9.1 while marks go up to 90. Without scaling, a column with big numbers can look more important only because of its size. For Logistic Regression, a `StandardScaler` was placed inside the model pipeline. It converts every column to a mean of 0 and a standard deviation of 1, and it learns this only from the training data. Random Forest does not need scaling.

Feature selection

All ten attributes were kept. The correlation analysis in 2.4 shows that every attribute has a positive link with placement (between 0.26 and 0.53), so none is useless. The ten columns were used as the model inputs and PlacementStatus as the target.

Data splitting

The dataset already comes as three separate files, so no random split was needed. train.csv is used for learning, validation.csv for choosing the model, and test.csv once at the end.

2.4 Exploratory Data Analysis (EDA)

All EDA below uses train.csv after cleaning (7,182 students).

Statistical analysis

```
stats = train[num_cols].describe().T[["count", "mean", "50%", "std", "min", "max"]]
stats = stats.rename(columns={"50%": "median"})
stats["mode"] = [train[c].mode()[0] for c in num_cols]
```

Table 7: Descriptive statistics of the numeric columns

Column	Mean	Median	Mode	Std dev	Min	Max
CGPA	7.70	7.7	6.5	0.64	6.5	9.1
Internships	1.04	1.0	1	0.66	0	2
Projects	2.02	2.0	3	0.87	0	3
Workshops/Certifications	1.01	1.0	0	0.90	0	3
AptitudeTestScore	79.41	80.0	90	8.14	60	90
SoftSkillsRating	4.32	4.4	4.8	0.41	3.0	4.8
SSC_Marks	69.13	70.0	55	10.44	55	90
HSC_Marks	74.40	73.0	84	8.89	57	88

- The average student has a CGPA of 7.70 and an aptitude score of about 79. For CGPA the mean and median are equal, so the values are spread evenly around the middle.
- SSC marks vary the most (standard deviation 10.44), CGPA the least (0.64).

- The mode is at the end of the range for several columns: aptitude 90 (the maximum), soft skills 4.8 (the maximum) and SSC marks 55 (the minimum). Many students sit at the scale limit, which is also visible in the histograms.
- Correlation with the target is covered with the heatmap (Figure 6).

Graphical analysis

Figure 1: Placed vs NotPlaced (bar chart)

4,192 students (58.4%) are NotPlaced and 2,990 (41.6%) are Placed. The two groups are not equal, but neither is very small, so the data can be used for modelling as it is. A model that always said NotPlaced would be right 58.4% of the time, so a useful model must beat that number.

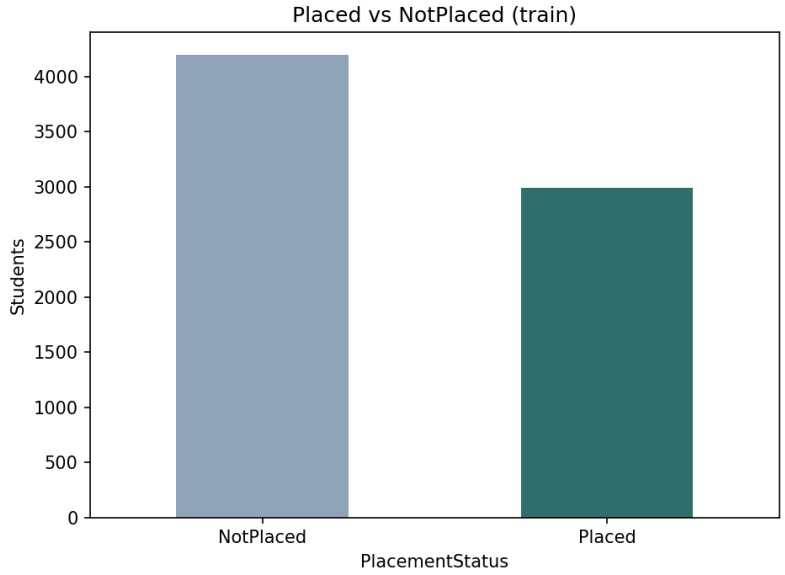


Figure 2: Spread of each column (histograms)

Most students have 1 internship and the most common number of projects is 3. Aptitude score, soft skills and SSC marks pile up at one end of their range (aptitude at 90, soft skills at 4.8, SSC at 55). HSC marks show two humps, one near 65 to 70 and one near 83 to 87, which suggests two groups of students with different results.

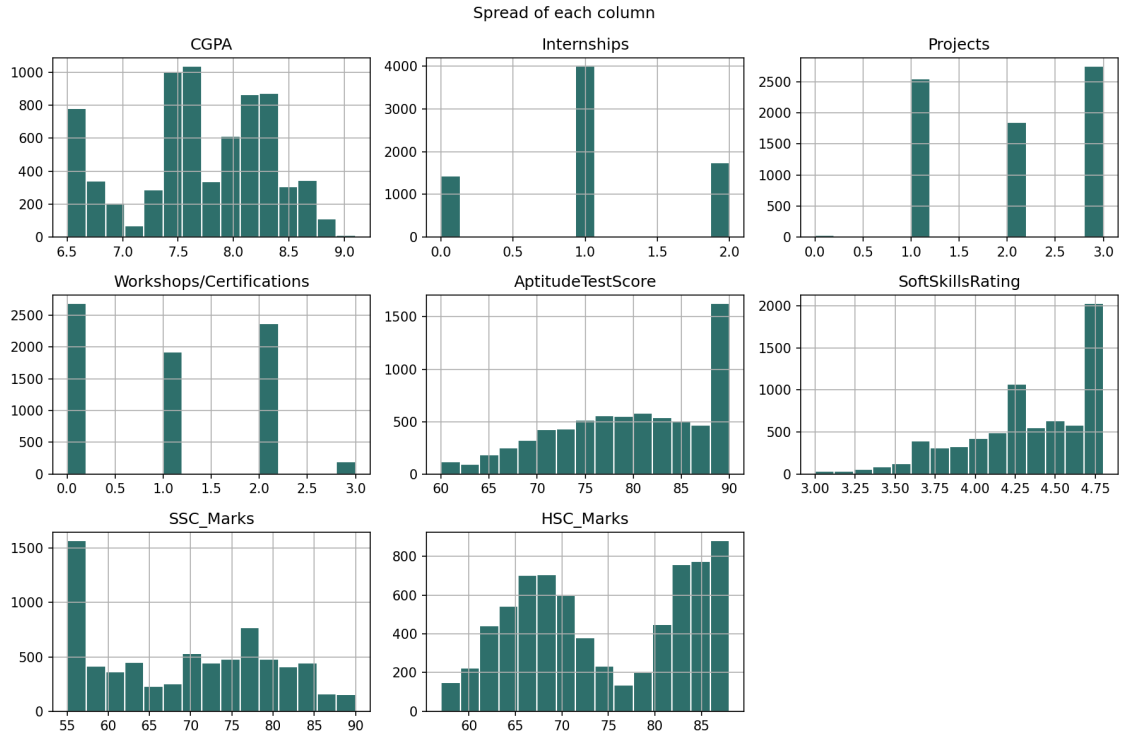


Figure 3: Placed vs NotPlaced for each column (box plots)

For every column the Placed group sits higher. Median CGPA is 8.2 for Placed and 7.5 for NotPlaced, aptitude score 86 and 76, soft skills 4.6 and 4.2, SSC marks 76 and 62, and HSC marks 83 and 69. The gap is largest for the two school marks and for aptitude. The dots below the Placed boxes are placed students with unusually low aptitude or soft skills scores, so a low score does not rule out placement.

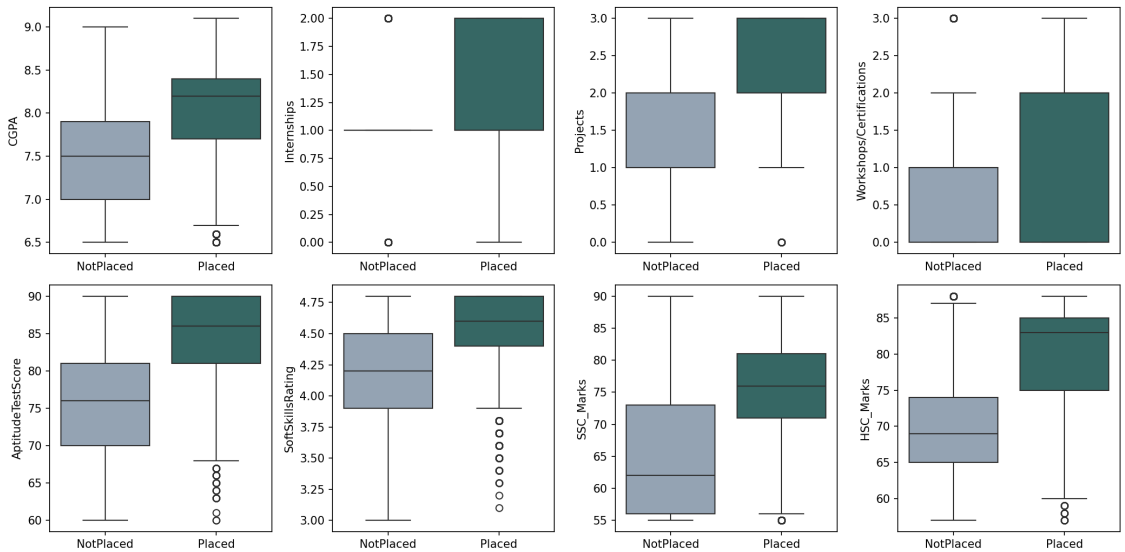


Figure 4: Placement rate for selected columns (bar charts)

Students with extracurricular activities are placed 61% of the time against 14% without, and with placement training 51% against 16%. Internships help mainly at 2 (70%); 0 and 1 internship give almost the same rate (33% and 32%). For projects, 3 projects gives the best rate (72%). The 64% for 0 projects comes from only 25 students, so it should not be trusted. In the first two charts, 0 means No and 1 means Yes. The exact values are in Table 8.

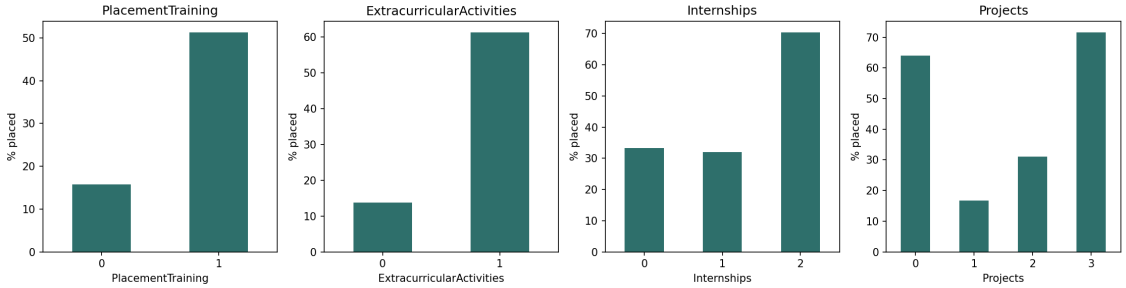


Table 8: Placement rate by group

Group	Placed (%)
No placement training / training	16 / 51
No extracurricular activities / activities	14 / 61
0 / 1 / 2 internships	33 / 32 / 70
0 / 1 / 2 / 3 projects	64 / 17 / 31 / 72

Figure 5: CGPA against aptitude score (scatter plot)

Placed students (dark) gather in the upper right, where both CGPA and aptitude score are high. Among students with CGPA 8.0 or more and aptitude 80 or more (2,213 students), 77% are Placed. Among students with CGPA below 7.5 and aptitude below 75 (990 students), only 8% are Placed. CGPA alone does not separate the two groups cleanly, because there are Placed and NotPlaced students at almost every CGPA value. Aptitude score adds a lot of information.

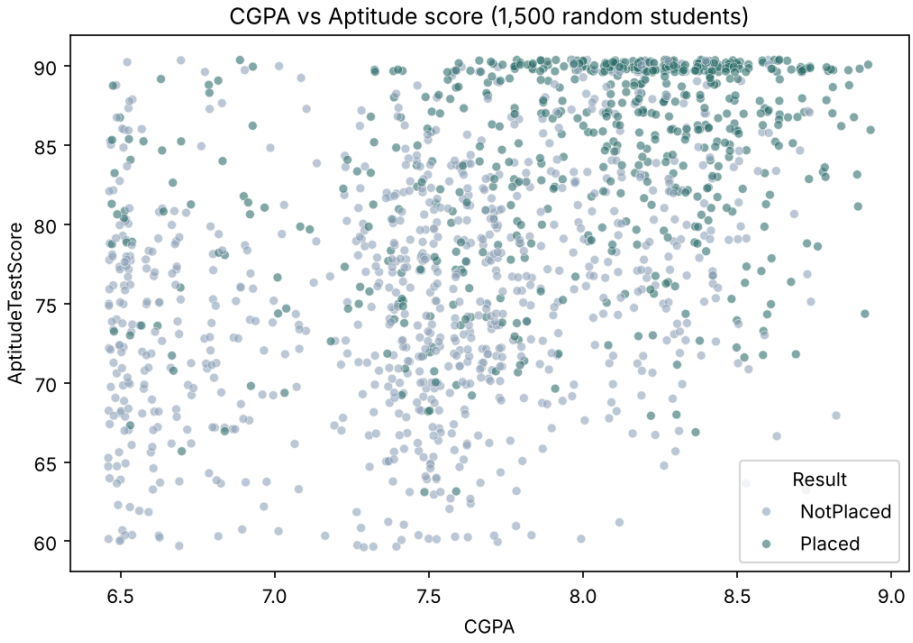
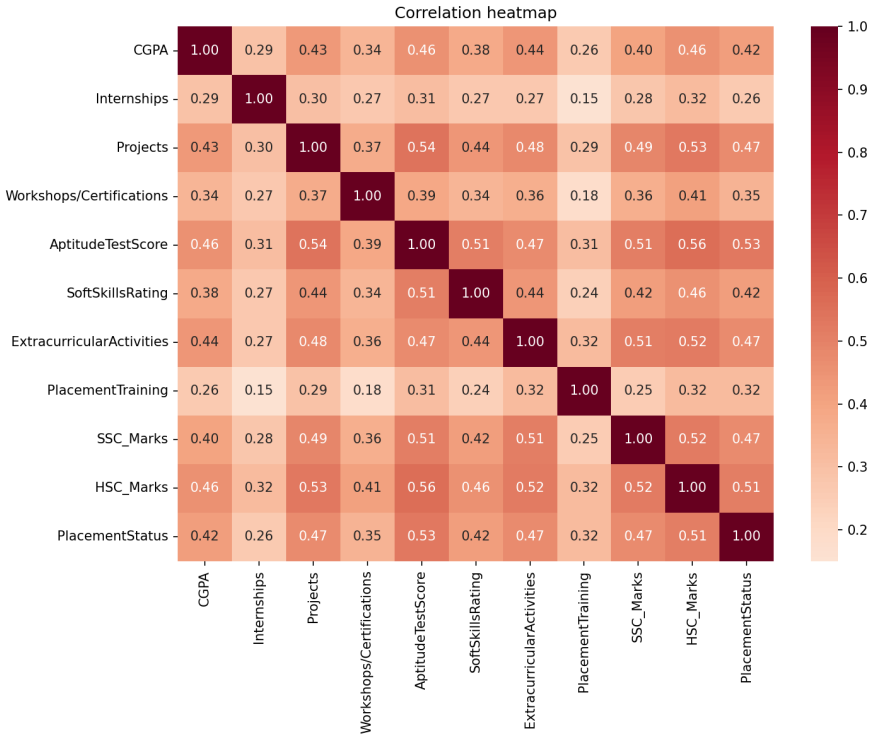


Figure 6: Correlation heatmap

Correlation is a number from -1 to 1 that shows whether two columns rise together. All columns have a positive link with PlacementStatus. The strongest are AptitudeTestScore (0.53) and HSC_Marks (0.51), followed by Projects, ExtracurricularActivities and SSC_Marks (0.47 each). The weakest are Internships (0.26) and PlacementTraining (0.32). The highest link between two input columns is 0.56 (HSC marks and aptitude score) and no pair is above 0.6, so no column needed to be removed. A link does not prove cause: for example, students with high marks may also be the ones who attend training.



2.5 Key Insights and Observations

1. **Aptitude score and school marks go with placement most.** AptitudeTestScore (0.53), HSC_Marks (0.51) and SSC_Marks (0.47) have the strongest links with PlacementStatus, and Placed students have clearly higher medians for all three (Figures 3 and 6).
2. **Activities and training make a big difference.** 61% of students with extracurricular activities are placed against 14% without, and 51% with placement training against 16% without (Figure 4).
3. **Internships matter mainly at the top.** Students with 2 internships are placed 70% of the time, but 0 and 1 internship give about the same rate (33% and 32%). Internships have the weakest link with placement (0.26).
4. **Strong students are strong on several things together.** With CGPA of 8.0 or more and aptitude of 80 or more, 77% are placed; with CGPA below 7.5 and aptitude below 75, only 8% (Figure 5).
5. **Placed students are the smaller group.** Only 41.6% of students are Placed, so a useful model must do better than 58.4%, the accuracy of always answering NotPlaced (Figure 1).
6. **The data is clean.** There are no missing values and no outliers. 43 repeated rows were removed from train. Several columns pile up at the ends of their scale (aptitude 90, soft skills 4.8, SSC 55), which looks like a limit of how the scores were recorded.

7. **No column is a duplicate of another.** The highest link between two input columns is 0.56, so all ten attributes can be used for modelling.

Limitations of the data

- The dataset page does not name the college, so the findings describe this dataset and may differ elsewhere.
- Correlation shows a link, not a cause.
- Our own student survey (responses.csv) was not merged with this dataset. It has no 10th and 12th marks and no Placed/NotPlaced result, so it cannot be joined to this data. It is kept as a separate study (Chapter 5).

Chapter 3: Model Building

3.1 Approach

The problem is a **classification** problem: for each student the model must choose between two classes, Placed and NotPlaced. The steps were:

1. Learn from train.csv (7,182 students after cleaning).
2. Compare two models on validation.csv (1,275 students) and choose the better one.
3. Check the chosen model once on test.csv (1,500 students it has never seen).
4. Save the model as a file and use it to give a placement chance for any student.

3.2 Inputs and Target

The model uses all ten attributes as inputs: CGPA, Internships, Projects, Workshops/Certifications, AptitudeTestScore, SoftSkillsRating, ExtracurricularActivities, PlacementTraining, SSC_Marks and HSC_Marks. The target is PlacementStatus (1 = Placed, 0 = NotPlaced). The three files were encoded in the same way (see 2.3).

```
target = "PlacementStatus"
features = [c for c in train.columns if c != target]
X_train, y_train = train[features], train[target]
X_val,   y_val   = val[features],   val[target]
X_test,  y_test  = test[features],  test[target]
```

3.3 The Two Models

- **Logistic Regression.** It gives each column one number (a weight). For a student it multiplies each value by its weight, adds everything up and converts the total into a chance between 0% and 100%. A StandardScaler puts all columns on the same scale first. This model is simple and easy to explain.
- **Random Forest.** It builds 200 decision trees, each asking a series of yes/no questions about the student, and combines their votes. It can learn complicated patterns, but it can also memorize the training data.

```
models = {
    "Logistic Regression": Pipeline([("scale", StandardScaler()),
                                     ("model", LogisticRegression(max_iter=1000))]),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
}
for name, m in models.items():
    m.fit(X_train, y_train)          # learning happens here
```

3.4 Training and Validation

Both models learned from train.csv only. They were then scored on train.csv and on validation.csv. Validation acts as a practice test, because the model has not learned from it.

Table 9: Accuracy of the two models

Model	Accuracy on train	Accuracy on validation
Logistic Regression	79.91%	80.00%
Random Forest	99.96%	79.37%

Random Forest is almost perfect on the students it learned from (99.96%) but not better on new students (79.37%). This means it memorized the training data, which is called **overfitting**. Logistic Regression scores about the same on both (79.91% and 80.00%), so what it learned also works on new students. Logistic Regression was therefore chosen. It also has the advantage that every column has one weight that can be read and explained.

3.5 Saving the Model

The chosen model was saved as one file, `placement_model.pkl`, together with the list of column names it expects.

```
joblib.dump({"model": best, "features": features, "name": best_name},
            "outputs/placement_model.pkl")
```

This file is the trained model. It can be loaded in any Python program to give a prediction for a new student without training again (see the Appendix).

3.6 Placement Chance and Readiness Bands

The model gives each student a chance of placement between 0% and 100%. Two simple rules were added to make the answer easy to read:

- If the chance is 50% or more, the model says **Placed**; otherwise **NotPlaced**.
- For the readiness label: below 40% is **Low**, 40% to 69% is **Medium** and 70% or more is **High**.

The 50% line and the three bands are our own choice for readability; the model itself only produces the percentage. Section 4.4 checks whether the bands are meaningful.

Chapter 4: Results and Evaluation

4.1 Test Accuracy

The chosen model (Logistic Regression) was checked once on test.csv, which it had never seen.

```
pred = best.predict(X_test)
print("Test accuracy:", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred, target_names=["NotPlaced", "Placed"]))
```

```
Test accuracy: 0.7913
              precision    recall  f1-score   support
   NotPlaced       0.82      0.82      0.82       870
      Placed       0.75      0.75      0.75       630
    accuracy                           0.79      1500
```

The model was right for 1,187 of 1,500 students, an accuracy of **79.1%**. This meets the target of 75% set in Objective 2 and beats the 58% obtained by always answering NotPlaced. The validation accuracy (80.0%) and the test accuracy (79.1%) are close, so the result is stable and not a lucky outcome of one file. The area under the ROC curve is about 0.87 (1.0 would be perfect, 0.5 would be guessing).

- **Precision for Placed (0.75):** when the model says Placed, it is right 75% of the time.
- **Recall for Placed (0.75):** the model finds 75% of the students who really got placed.
- NotPlaced students are found slightly better (0.82 on both) than Placed students.

4.2 Confusion Matrix

Figure 7: Right and wrong guesses on the test file (confusion matrix)

Of the 870 students who were really NotPlaced, the model found 715 and wrongly called 155 of them Placed. Of the 630 who were really Placed, it found 472 and wrongly called 158 of them NotPlaced. The two kinds of mistake are almost equal in number (155 and 158), so the model is not biased towards one answer. The numbers are also shown in Table 10.

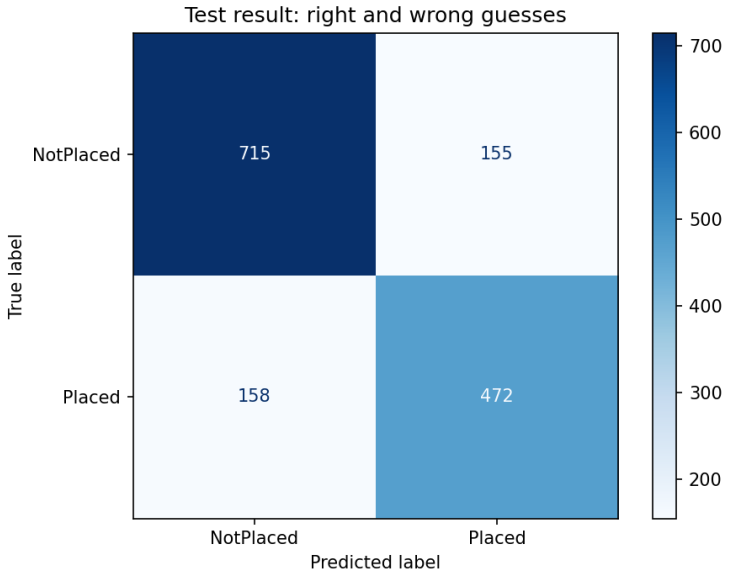


Table 10: Confusion matrix on the test file

	Model said NotPlaced	Model said Placed
Really NotPlaced (870)	715 correct	155 wrong
Really Placed (630)	158 wrong	472 correct

4.3 What the Model Learned

Figure 8: How much each column matters to the model

The importance is each column's weight in Logistic Regression, shown as a percentage of the total. Aptitude score has the biggest influence (21.5%), followed by placement training and extracurricular activities. School marks (SSC and HSC), soft skills and CGPA each add about 9% to 11%. This agrees with the EDA, where aptitude score had the strongest correlation with placement (Figure 6).

Internships have almost no weight (0.8%), although Figure 4 shows that students with 2 internships are placed more often (70%). One reason may be that 0 and 1 internship have the same placement rate (33% and 32%), so a single straight-line weight cannot describe the pattern. Internships are also linked to other columns such as projects (0.30), so part of their effect may be taken by those columns. This is a possible explanation and was not tested further.

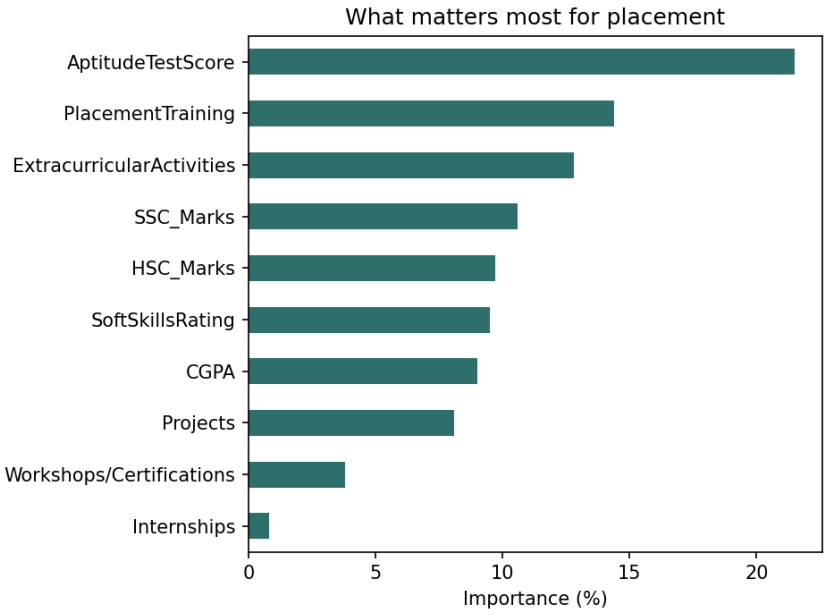


Table 11: Importance of each column

Column	Importance (%)
AptitudeTestScore	21.5
PlacementTraining	14.4
ExtracurricularActivities	12.8
SSC_Marks	10.6
HSC_Marks	9.7
SoftSkillsRating	9.5
CGPA	9.0
Projects	8.1
Workshops/Certifications	3.8
Internships	0.8

4.4 Is the Readiness Percentage Meaningful?

A percentage is only useful if students with a high percentage are really placed more often. The 1,500 test students were grouped by the model's chance of placement and the real placement rate in each group was checked.

Table 12: Real placement rate in each readiness band (test file)

Readiness band	Model chance	Students	Really placed (%)
Low	below 40%	777	14.8
Medium	40% to 69%	277	54.2
High	70% or more	446	81.8

The real placement rate rises clearly from Low (14.8%) to Medium (54.2%) to High (81.8%). This means the percentage works as a readiness score: a student in the High band is far more likely to be placed than a student in the Low band. The Medium band is the least certain group, which is expected because those students are close to the 50% line.

4.5 Example Predictions

Table 13 shows ten test students picked at random (the full list of 1,500 is saved in `table_test_predictions.csv`).

Table 13: Ten random test students

CGPA	Intern-ships	Projects	Aptitude	Training	Chance (%)	Model says	Real result	Right?
8.2	1	3	88	Yes	82.9	Placed	NotPlaced	No
7.7	1	3	90	Yes	84.0	Placed	Placed	Yes
8.0	2	3	90	Yes	90.0	Placed	Placed	Yes
6.5	1	2	78	Yes	24.0	NotPlaced	NotPlaced	Yes
6.8	1	3	83	No	39.0	NotPlaced	NotPlaced	Yes
8.2	1	2	88	Yes	83.0	Placed	Placed	Yes
8.4	2	3	90	Yes	90.0	Placed	Placed	Yes
6.6	1	1	87	No	9.0	NotPlaced	NotPlaced	Yes
7.6	0	2	74	Yes	49.9	NotPlaced	Placed	No
8.5	1	3	90	Yes	90.0	Placed	Placed	Yes

8 of the 10 are correct. The two wrong ones are borderline or unusual cases: one student with a very high chance (82.9%) who was not placed, and one at 49.9%, just below the 50% line, who was placed. A model cannot be right for every student, because placement also depends on things that are not in the data.

The saved model was also tried on two made-up students (Table 14).

Table 14: Two made-up students

Input	Strong student	Weak student
CGPA / aptitude score	8.5 / 88	6.6 / 62
Internships / projects / workshops	2 / 3 / 2	0 / 1 / 0
Soft skills rating	4.6	3.5
Activities / placement training	Yes / Yes	No / No
SSC / HSC marks	85 / 85	55 / 60
Placement chance	**90.1% (High)**	**0.5% (Low)**

4.6 Summary of Results

Table 15: Summary of results

Item	Result
Chosen model	Logistic Regression
Validation accuracy	80.0%
Test accuracy (unseen students)	79.1%
Baseline (always NotPlaced)	58%
Most important column	AptitudeTestScore (21.5%)
Real placement rate: Low / Medium / High band	14.8% / 54.2% / 81.8%

Chapter 5: Our Own Student Survey

5.1 What We Collected

Alongside the public dataset, we collected our own data with a Google Form filled in by senior students (60 responses). The form asked about CGPA (as a band), programming languages known, coding problems solved, hackathons, internships, technical clubs, aptitude level, mock interviews, communication confidence, target companies, salary expectation, expected domain and resume quality. The aim was to describe how ready students are and to compare our college with the public data.

The responses were cleaned in three steps (files `step1_apeksha.py`, `step2_ananya.py` and `step3_ashritha.py` in the first repository):

1. Almost empty responses were dropped, leaving **57** usable students. Answers typed as free text (for example "300+" or "50-60") were converted to numbers, and very large counts were capped so a single student could not distort the graphs.
2. Words were turned into numbers (for example aptitude Weak, Moderate, Strong into 1, 2, 3; CGPA bands into their middle values).
3. A **readiness index** from 0 to 100 was calculated as the average of seven scaled indicators: coding problems solved, hackathons, mock interviews, aptitude, communication, resume quality and internship. Students were labelled Low (33 or below), Medium (above 33 up to 50) or High (above 50).

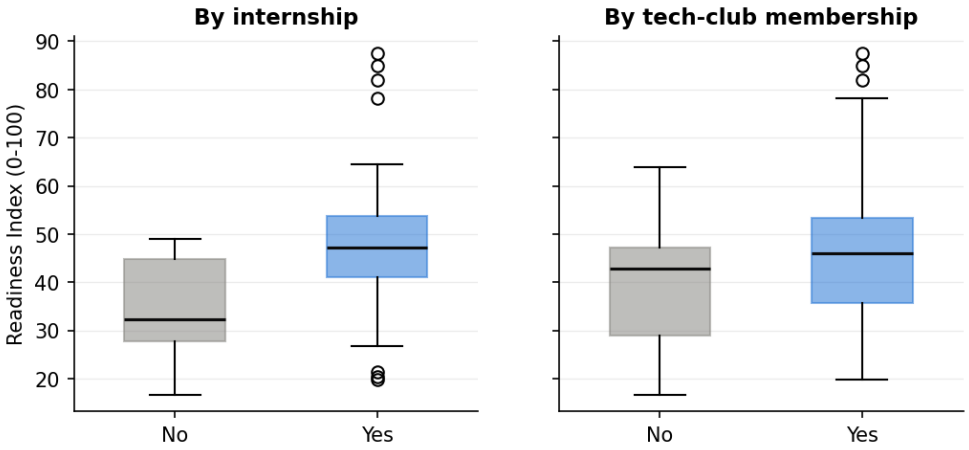
5.2 What the Survey Shows

Table 16: Survey summary (57 students)

Item	Result
Readiness level	14 Low, 28 Medium, 15 High
Readiness index	average 44.7, middle value (median) 45.4, range 16.7 to 87.6
Did an internship	40 students (70%)
Member of a technical club	37 students (65%)
Aptitude level	41 Moderate, 7 Weak, 5 Strong, 4 not practiced yet
Communication	36 Moderate, 18 Very confident, 3 Low confidence
No hackathons / no mock interviews	36 students (63%) for each
CGPA band	23 in 9.0 to 10.0, 17 in 8.0 to 9.0, 15 in 7.0 to 8.0, 2 in 6.0 to 7.0

Figure 9: Readiness index by internship and by technical club membership

Students who did an internship have a clearly higher median readiness (47.2) than those who did not (32.3). Club members are slightly higher (46.1) than non-members (42.8), a smaller gap. Most students are in the Medium group and about 63% have attended no hackathon and no mock interview, which shows where preparation is weakest. With only 57 students these are rough indications, not firm conclusions.



5.3 Why the Survey Was Not Used with the Model

We first planned to use the survey as unseen test data for the model. This turned out not to be possible, for three reasons:

- **Different columns.** The public data has 10th and 12th marks, workshops and placement training. Our form does not. The model uses these columns, and 10th and 12th marks are important for placement. Filling them with average values would make the model's answers unreliable.
- **No placement result.** Our form was filled by students who are not yet placed, so there is no Placed or NotPlaced answer to compare with. Accuracy cannot be measured.
- **Different meaning.** The readiness index is a score we built ourselves from our own indicators. It is not the same thing as the placement chance from the model.

The survey is therefore kept as a separate study. The model was trained, validated and tested only on the public dataset, where the real results are known.

Chapter 6: Conclusion, Limitations and Future Work

6.1 Conclusion

The project cleaned and explored a public placement dataset of 10,000 students and built a model that predicts placement from ten student details.

- **Objective 1 (explore the data):** the data had no missing values or outliers, and 43 repeated rows were removed. Aptitude score, 10th and 12th marks, projects, extracurricular activities and placement training had the strongest links with placement, while internships had the weakest.
- **Objective 2 (predict placement):** Logistic Regression was chosen over Random Forest because Random Forest memorized the training data. On 1,500 unseen test students the chosen model reached **79.1% accuracy**, above the 75% target and well above the 58% baseline.
- **Readiness percentage:** the model's chance of placement behaves like a readiness score. The real placement rate was 14.8% in the Low band, 54.2% in the Medium band and 81.8% in the High band.

The trained model is saved as one file (`placement_model.pkl`), and the whole work is repeatable from one notebook kept on GitHub.

6.2 Limitations

- The dataset page does not name the college, so the model describes this dataset and may not fit another college without checking.
- The model uses ten columns. Skills such as coding practice, hackathons or mock interviews are not in the data, so the model cannot use them.
- About 1 in 5 students (20.9%) is predicted wrongly. The model gives a chance, not a promise, and should not be used to judge a real student.
- The results show links, not causes. For example, placement training goes with higher placement, but this does not prove the training itself causes it.
- Our own survey could not be used for testing, because it lacks 10th and 12th marks and has no placement result (see 5.3).
- The 50% line and the Low, Medium and High bands are our own choice for readability.

6.3 Future Work

1. Collect data from our own college that includes 10th and 12th marks and the real placement result, so the model can be tested on our students.
2. Add skill columns from the survey, such as coding problems solved, hackathons and mock interviews, and check whether they improve accuracy.
3. Try and tune other models, for example gradient boosting, and compare them with Logistic Regression in the same way.
4. Turn the saved model into a small web page where a student types their details and sees the readiness percentage.
5. Study internships more closely, since the model gave them almost no weight while students with 2 internships were placed more often.

Appendix: Project Files, How to Run and How to Use the Model

A. Project Files on GitHub

The project is in the repository **Placement-Model** (github.com/apekshacbhat4-art/Placement-Model). The survey work is in the separate repository **Student-Placement-Analysis** (github.com/apekshacbhat4-art/Student-Placement-Analysis).

Table 17: Files in the Placement-Model repository

Item	What it is
data/	train.csv, validation.csv and test.csv from the public dataset
placement_model.ipynb	The notebook: load, clean, explore, train, validate, test, save
fig/	The graphs used in this report
output/	The tables and the trained model placement_model.pkl
README.md, LICENSE	Short description and licence of the repository

B. How to Run the Notebook

1. Open Google Colab (colab.research.google.com), choose File, Open notebook, the GitHub tab, and select placement_model.ipynb from the Placement-Model repository.
2. In the first code cell, set REPO_URL to the repository address so the data folder is copied into Colab.
3. Click Runtime, then Run all. The notebook runs in about a minute.
4. The last cell packs all graphs, tables and the model into results.zip and downloads it.

The notebook also has a cell with sliders where any student's details can be typed in and the placement chance is shown.

C. How to Use the Saved Model

```
import joblib, pandas as pd

saved = joblib.load("placement_model.pkl")     # the file saved by the notebook
student = pd.DataFrame([{
    "CGPA": 8.0, "Internships": 1, "Projects": 2, "Workshops/Certifications": 1,
    "AptitudeTestScore": 78, "SoftSkillsRating": 4.2, "ExtracurricularActivities": 1,
    "PlacementTraining": 1, "SSC_Marks": 70, "HSC_Marks": 72}])[saved["features"]]

chance = saved["model"].predict_proba(student)[0][1] * 100
print(f"Chance of placement: {chance:.1f}%")
print("Prediction:", "Placed" if chance >= 50 else "NotPlaced")
```

Example output: chance 44.5%, prediction NotPlaced. Give Yes/No columns as 1/0, as in the notebook. Load the file with a similar scikit-learn version (e.g. in Colab).

D. Tools Used and Data Source

Python (pandas, NumPy, matplotlib, seaborn, scikit-learn, joblib) in Google Colab; code on GitHub. Data: Krooz/Campus_Recruitment_CSV on Hugging Face.